

React.js, Node.js, MongoDB ve API Entegrasyonlarını Kullanan Akademik Personel İşe Alım Sistemi

FRANCKY RONSARD SAAH
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
francky877832@gmail.com

KHADIM DIEYE
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
Khadimd978@gmail.com

TURAN ASGARLI
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
turanaskerov2004@gmail.com

Özet- Akademik personelin işe alım süreci, başvuruları, değerlendirmeleri ve onayları ele almak için verimli ve güvenilir bir sistem gerektiren kritik bir görevdir. Bu proje, modern web teknolojilerini kullanarak başvuru sürecini kolaylaştıran web tabanlı bir akademik personel işe alım sistemi sunmaktadır. Sistem, ön uç için React.js, arka uç için Node.js (Express.js) ve veritabanı olarak MongoDB kullanılarak geliştirilmiştir. Ayrıca, dosya depolama için Amazon S3, bildirimler için Twilio ve Brevo ve güvenlik için JWT tabanlı kimlik doğrulama gibi üçüncü taraf hizmetleri entegre eder.

Bu sistemin önemli bir özelliği, başvuru sahiplerinin kimlik bilgilerini doğrulamak ve doğrulama sürecini kolaylaştırmak için devlet API'leri (e-Devlet ve Nüfus Müdürlüğü) ile entegre olmasıdır. Sistem, başvuruları yönetmek, adayları değerlendirmek ve işe alım kararlarını sonuçlandırmak için her biri belirli işlevlere sahip başvuru sahipleri, yöneticiler, müdürler ve jüri üyeleri dahil olmak üzere birden fazla kullanıcı rolü sağlar.

Bu makale sistemin mimarisini, uygulamasını ve işlevlerini detaylandırmaktadır. Sonuçlar, platformun idari yükü önemli ölçüde azalttığını, başvuru takibini iyileştirdiğini ve işe alım sürecinin verimliliğini artırdığını göstermektedir. Gelecekteki iyileştirmeler arasında yapay zeka odaklı aday sıralaması ve gelişmiş güvenlik mekanizmaları yer alabilir.

Anahtar Kelimeler: Akademik İşe Alım, React.js, Node.js, MongoDB, API Entegrasyonu, Kimlik Doğrulama, AWS S3, Twilio, Brevo

I. GİRİŞ

1. Arka plan

Akademik personelin işe alımı, üniversitelerde eğitim ve araştırma kalitesinin sağlanmasında hayati bir rol oynamaktadır. Geleneksel olarak bu süreç, başvuruların manuel olarak gönderilmesini, belge doğrulamasını ve seçim komiteleri tarafından değerlendirilmesini içermekte ve gecikmeler, belgelerin yanlış yönetimi ve şeffaflık eksikliği gibi verimsizliklere yol açmaktadır. Akademiye dijital dönüşüme duyulan ihtiyacın artmasıyla birlikte, bu süreci kolaylaştırırken güvenlik, ölçeklenebilirlik ve resmi devlet veritabanlarıyla entegrasyon sağlayan otomatik bir sisteme ihtiyaç duyulmaktadır.

2. Hedefler

Bu projenin temel amacı, kurumların başvuruları dijital olarak yönetebilmelerini sağlayan web tabanlı bir akademik işe alım sistemi geliştirmektir:

- Akademik pozisyonlar için sorunsuz başvuru gönderimi.
- Başvuru sahiplerinin kimlik bilgilerinin e-Devlet ve Nüfus Müdürlüğü API'si kullanılarak otomatik olarak doğrulanması.
- Amazon S3 kullanarak güvenli belge depolama.
- Çoklu rol sistem ile farklı işlevler başvuru sahipleri, idareciler, yöneticiler ve jüri üyeleri için.
- Tarafından desteklenen bildirim ve iletişim sistemi Twilio ve Brevo.
- JWT kullanarak kimlik doğrulama ve erişim kontrolü.

3. Sorun Bildirimi

Birçok akademik kurumdaki mevcut işe alım süreçleri çeşitli zorluklarla karşı karşıyadır:

- Uygulamaların manuel olarak ele alınması nedeniyle zaman alan ve verimsiz iş akışları.
- Kayıp veya yanlış yerleştirilmiş kimlik bilgilerine yol açan yanlış belge yönetimi.
- Gerçek zamanlı güncellemelerin olmaması, adayların başvuru durumlarını sık sık sorgulamalarını gerektiriyor.
- Harici doğrulama sistemleriyle sınırlı entegrasyon, başvuru sahibinin kimlik bilgilerinin verimli bir şekilde doğrulanmasını zorlaştırır.
- Hassas kişisel ve akademik verilerin işlenmesinde güvenlik riskleri.

Bu proje, işe alım sürecinin verimliliğini artıran modern, güvenli ve ölçeklenebilir bir işe alım platformu uygulayarak bu sorunları ele almaktadır.

4. Katkılar

Bu sistem akademik işe alım sürecine şu şekilde katkıda bulunur:

- Tüm işe alım iş akışını otomatikleştirerek idari yükü azaltır.
- Başvuru sahipleri, yöneticiler ve değerlendiriciler için merkezi bir platform sağlanması.

- Kimlik doğrulama ve güvenli depolama yoluyla başvuru sahibi verilerinin **güvenliğinin** ve **bütünlüğünün sağlanması**.
- **Güçlendirme Şeffaflık** tarafından sağlayan gerçek zamanlı uygulama durum güncellemeleri.
- Devlet sistemleriyle API entegrasyonu yoluyla **belge doğrulamayı kolaylaştırmak**.

5. Makalenin Organizasyonu

Bu makalenin geri kalanı aşağıdaki şekilde yapılandırılmıştır:

- **Bölüm II**'de sistem mimarisi ve kullanılan teknolojiler açıklanmaktadır.
- **Bölüm III**, ön uç, arka uç, veritabanı, kimlik doğrulama ve üçüncü taraf entegrasyonlarını detaylandırarak sistem bileşenlerini sunmaktadır.
- **Bölüm IV**'te sistemin işlevleri ve kullanıcı rolleri tartışılmaktadır.
- **Bölüm V**, temel kod bölümleri de dahil olmak üzere uygulama ayrıntılarına ilişkin bilgiler sunmaktadır.
- **Bölüm VI**'da sözlük sunulmaktadır.
- **Bölüm VII** makaleyi sonlandırmakta ve gelecekteki çalışmaları özetlemektedir.
- **Bölüm VII**'de sonuçlar sunulmaktadır.

II. SİSTEM MİMARİSİ

1. Genel Bakış

Akademik Personel İşe Alım Sistemi, daha önce de belirtildiği gibi modern, ölçeklenebilir ve güvenli bir mimariyi takip eden web tabanlı bir platform olarak tasarlanmıştır. Ön ucun (**React.js**) RESTful API aracılığıyla arka ucla (**Node.js/Express**) etkileşime girdiği bir istemci-sunucu modeli kullanılarak geliştirilmiştir. Arka uç, veri kalıcılığı için bir **MongoDB veritabanı (noSQL)** ile iletişim kurar ve **dosya depolama için Amazon S3**, **bildirimler için Twilio** ve **kimlik doğrulama için e-Devlet ve Nüfus Müdürlüğü API'leri** gibi harici hizmetlerle entegre olur.

Sistem aşağıdaki mimari hususlar göz önünde bulundurularak inşa edilmiştir:

- **Ölçeklenebilirlik:** Modüler mimari, sistemin verimli bir şekilde ölçeklenmesini sağlar.
- **Güvenlik:** Kimlik doğrulama, yetkilendirme ve güvenli API iletişimlerini sağlar.
- **Entegrasyon:** Doğrulama ve bildirimler için harici API bağlantılarını destekler.
- **Kullanıcı Rol Yönetimi:** Farklı erişim seviyelerine sahip birden fazla kullanıcı türünü yönetir.

2. Kullanılan Teknolojiler

Sistem, aşağıdaki temel teknolojileri içeren tam yığın bir JavaScript ekosistemine dayanır:

	Kullanılan Teknoloji	Amaç
Ön Uç	React.js, Bootstrap	Kullanıcı arayüzü tasarımı

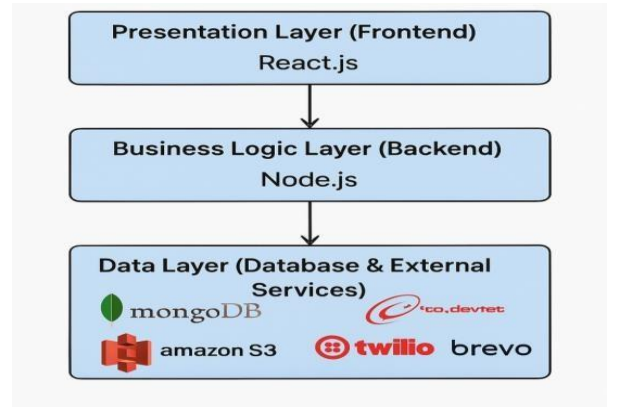
Arka uç	Node.js, Express.js	API geliştirme ve iş mantığı
Veritabanı	MongoDB	Veri depolama ve yönetimi
Kimlik Doğrulama	JSON Web Belirtecisi (JWT)	Güvenli kullanıcı kimlik doğrulaması
Dosya Depolama	Amazon S3	Güvenli belge yükleme ve alma
Bildirimler	Twilio API,	Kullanıcılar için SMS/E-posta uyarıları
	Brevo API, Yerleşik Bildirim	Kullanıcılar için e-posta uyarıları
API Entegrasyon	e-Devlet & Nüfus Müdürlüğü	Kimlik ve belge doğrulama

Tablo 1. Kullanılan teknolojiler

3. Sistem Mimarisi Diyagramı

Sistem, aşağıdakilerden oluşan üç katmanlı bir mimariyi takip eder of:

- **Sunum Katmanı (Ön Uç):** React.js kullanıcı arayüzünü yöneterek kullanıcıların sistemle etkileşime girmesini sağlar.
- **İş Mantığı Katmanı (Arka Uç):** Node.js istekleri işler, kimlik doğrulama uygular ve harici API'lere bağlanır.
- **Veri Katmanı (Veritabanı ve Harici Hizmetler):** MongoDB kullanıcı ve uygulama verilerini depolarken, harici API'ler doğrulama (E-devlet), dosya depolama (Amazon S3) ve bildirimleri (Twilio, Brevo) işler.



Şekil1. Sistem Genel Mimarisi

4. Etkileşim Akışı

Sistem aşağıdaki şekilde çalışır:

- A. **Ön Uç (React.js & Bootstrap) - Kullanıcı Kontrolü, Kayıt ve Kimlik Doğrulama**

Aşama 1: Token Doğrulama ve Kullanıcı Yeniden Yönlendirme

- **Belirteç Kontrolü:** Bir kullanıcı sisteme eriştiğinde, uygulama ilk olarak tarayıcının yerel depolama alanında geçerli bir JWT belirtecinin varlığını kontrol eder.
- **Otomatik Giriş:** Geçerli bir token bulunursa, kullanıcı otomatik olarak oturum açar ve 7 gün daha geçerlilik süresi olan yeni bir JWT token oluşturulur. Bu işlem, kullanıcıların token'ın geçerlilik süresi içinde sistemi her ziyaret ettiklerinde oturum açmalarına gerek kalmamasını sağlar.
- **Belirteç Bulunamadı:** Geçerli bir belirteç yoksa veya kullanıcı sistemi ilk kez ziyaret ediyorsa, kullanıcı kayıt işlemine başlamak için **ControlUser** sayfasına yönlendirilir.

Aşama 2: Kullanıcı Veri Girişi ve e-Devlet API Doğrulaması

- **Kişisel Bilgi Gönderimi:** **ControlUser** sayfasında, kullanıcıların **TC Kimlik, Ad, Soyad ve Doğum Yılı** gibi temel kişisel bilgilerini girmeleri gerekmektedir.
- **SOAP API Çağırısı:** Girilen veriler arka uca gönderilir ve arka uç da kullanıcı doğrulaması için **e-Devlet API**'sini sorgulamak üzere **SOAP protokolünü** kullanır. Bu adım, adayın devlet veri tabanında var olmasını sağlar.
- **Boolean Yanıt:** Arka uç, kullanıcının var olup olmadığını gösteren bir boolean döndürür. Kullanıcı bulunamazsa, kullanıcıya bir hata mesajı görüntülenir.
- **Doğrulanmış Kullanıcı Yönlendirme:** Kullanıcı e-Devlet sisteminde bulunursa, kaydını tamamlamak için giriş sayfasına yönlendirilir.

Aşama 3: Kullanıcı Kaydı ve Kimlik Doğrulama

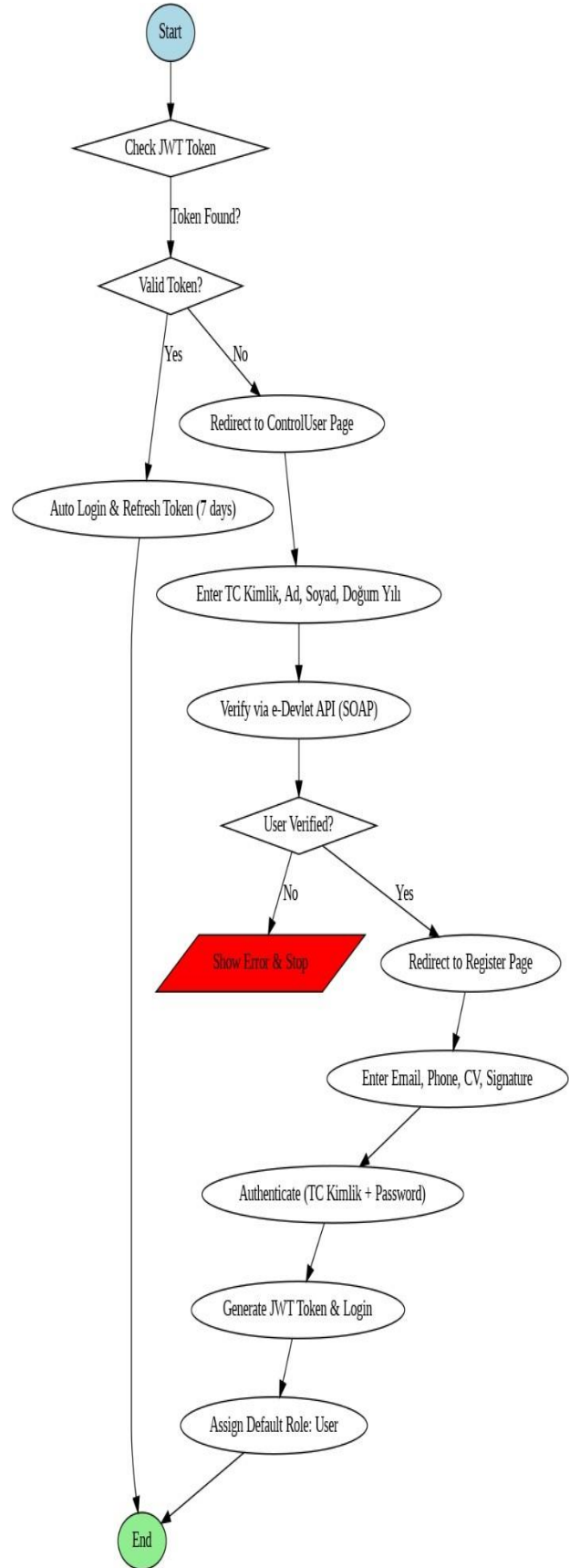
- **Kaydın Tamamlanması:** Giriş sayfasında, kullanıcılardan **e-posta, telefon numarası, özgeçmiş** ve başvuru formlarında kullanılacak bir **imza resmi** gibi ek kayıt bilgilerini vermeleri istenir.
- **Kimlik Doğrulama Süreci:** Kullanıcı bu bilgileri gönderdikten sonra **TC Kimlik** ve **parola** kullanılarak **kimlik** doğrulaması yapılır. Bu noktada yeni bir JWT token oluşturulur ve kullanıcı otomatik olarak oturum açar.

Aşama 4: Rol Atama ve Erişim Kontrolü

- **Varsayılan Kullanıcı Rolü:** Başlangıçta, tüm yeni kullanıcılara varsayılan **kullanıcı** rolü atanır.

Not : Rol Yönetimi: Yalnızca sistem **yöneticisinin** kullanıcı rollerini değiştirme yetkisi vardır, bu da kullanıcıların gereksinimlere göre **yönetici, jüri** veya **müdür** gibi rollere yükseltilmesine olanak tanır. Bu rol tabanlı erişim kontrolü, kullanıcıların sistem içinde uygun izinlere sahip olmasını sağlar.

Yukarıdaki sürecin akış şeması burada olabilir:



şekil2. Kimlik Doğrulama Süreci

B. Arka Uç (Node.js & Express.js) - Uygulama Gönderimi

Başvuru gönderme süreci sistemin kritik bir bileşenidir ve tüm aday bilgilerinin doğru ve güvenli bir şekilde girilmesini sağlayarak akademik pozisyonlar için başvuruyu kolaylaştırmak üzere tasarlanmıştır. Ayrıca gerekli belgelerin sunulmasını ve daha sonra jüri tarafından değerlendirilmek üzere saklanmasını sağlar.

Aşama 1: İş Duyurularına Erişim ve Giriş/Kayıt

Ana sayfa: Ana sayfa, pozisyon türüne göre kategorize edilmiş en son akademik iş ilanlarını sunar: **Yardımcı Doçent, Doçent, Profesör ve Tümü.**

Giriş/Kayıt: Kullanıcı oturum açmamışsa, oturum açma veya kayıt sayfasına yönlendirilir. Başarılı bir girişten sonra, iş ilanının tamamı görüntülenir. Kullanıcı daha sonra devam etmek için -Başvur|| düğmesine tıklayabilir.

2. Aşama: Dinamik Çok Adımlı Başvuru Formu

- **Başvuru Formu Oluşturma:** "Başvur" düğmesine tıkladıktan sonra kullanıcı çok adımlı bir başvuru formuna yönlendirilir. Form, her bir belge olarak saklandığı *MongoDB "forms" koleksiyonundan* dinamik olarak oluşturulur. Her bir form *KOÜ Atama Yönetmeliği Tablo 5'teki* kategorilere karşılık gelmektedir.

- **Form Yapısı:** Form, *KOÜ Atama Yönetmeliği Tablo 5'te yer alan* farklı kategorileri temsil eden 12 adımdan oluşmaktadır.

Her adımda kategoriye belirten bir başlık (H1) (örn. **A Makaler**) ve ardından bu kategorideki uygun faaliyeti seçmek için bir açılır menü (örn. A Faaliyeti için 1'den 8'e kadar) bulunmaktadır.

- Formun Doldurulması:

Bir faaliyet seçildiğinde, adaydan gerekli bilgileri doldurmasını ve söz konusu faaliyete ilişkin kanıtlayıcı belgeleri yüklemesini isteyen boş bir form görüntülenir. Bu belgeler jüri tarafından verilen bilgilerin geçerliliğini doğrulamak için kullanılacaktır.

Adaylar farklı kategorilerden istedikleri kadar faaliyet doldurabilirler ancak her kategori için aynı anda yalnızca bir faaliyet sunabilirler.

- Adımlar Arasında Gezinme:

Bir adımı tamamladıktan sonra, aday formda gezinmek için "Sonraki" veya "Önceki" ye tıklayabilir. Form esneklik sağlayarak kullanıcıların gerektiği kadar faaliyet doldurmasına olanak tanır.

3. Aşama: İnceleme ve PDF Oluşturma

- **İnceleme Sayfası:** Form doldurulduktan sonra kullanıcı "İnceleme" sayfasına yönlendirilir.

Tamamlanan tüm faaliyetlerin, *KOÜ Atama Sonuçları Tablosu 4'e* göre hesaplanan puanların ve diğer faaliyetlerin yer aldığı bir PDF belgesi otomatik olarak oluşturulur.

adayın imzası bir resim olarak (kayıt sürecinden).

Adaya ayrıca jüri için değerlendirme esasını (**Puanlanan Faaliyet Dönemi**) seçmesi için dört radyo düğmesi sunulur.

- PDF Özelleştirme:

Oluşturulan PDF, aday tarafından doldurulan faaliyetleri, hesaplanan puanları ve adayın imzasını içerir. Sistem ayrıca kullanıcının başvuruyu sonuçlandırmadan önce formdaki herhangi bir veriyi gözden geçirme ve değiştirme seçeneğini de içerir.

4. Aşama: Teslim ve Başvuru Takibi

- Başvurunun Sonuçlandırılması:

Aday PDF'deki ayrıntıları inceledikten ve verilen bilgilerden tatmin olduktan sonra

Nihai belgeyi oluşturmak için -PDF Oluştur|| düğmesine .

PDF oluşturulduktan sonra aday Başvurularını jüri tarafından incelenmek üzere göndermek için -Başvuru Gönder|| düğmesine basınız.

- Başvuru Durumunun Takibi:

Aday daha sonra, başvurusunun durumunu istediği zaman takip edebileceği **Başvuruyu Görüntüle** sayfasına yönlendirilir. Bu sayfada, başvurularının oluşturulan PDF belgesini de indirebilirler.

- **Dosya Depolama:** Adayın prova dosyaları ve nihai oluşturulan PDF dahil olmak üzere tüm belgeler **Amazon S3'te** güvenli bir şekilde saklanır ve yetkili personel tarafından kolay erişim ile güvenli ve güvenilir depolama sağlanır.

C. Başvuru Atama Süreci - Yönetici

Bir aday başvurusunu gönderdikten sonra, bir yönetici incelenmek üzere jüri üyelerine atayana kadar durumu "beklemede" kalır. Bu süreç, verimli çalışma için üç bölüme ayrılmış olan **Yönetici Paneli** aracılığıyla yönetilir.

Bölüm 1: Duyuru Yönetimi

Yönetici panelinin ilk bölümü, yöneticinin iş ilanlarını yönetmesini sağlar. Bu bölüm şunları içerir:

- **Duyuru Listesi:** Yönetici tarafından gönderilen tüm duyurular burada görüntülenir.

- **Duyuruları Değiştirin:** Yönetici her bir duyurunun ayrıntılarını değiştirebilir.

- **Sıralama Seçenekleri:** Duyurular şu şekilde sıralanabilir:

- **Devam ediyor:** Şu anda aktif olan duyurular.
- **Geçmiş Duyurular:** Halihazırda kapanmış olan duyurular.
- **Yaklaşan Duyurular:** Henüz başvuru portalında yer almayan duyurular.

Bölüm 2: Uygulama Ataması

Yönetici panelinin ikinci bölümü jüri üyelerine atanmayı bekleyen başvuruları gösterir. Bu bölüm şunları içerir:

- **Bekleyen Başvurular:** Bir seferde atanmayı bekleyen beş başvuruyu görüntüler.
- **"Diğer Başvurular" Düğmesi:** Bu düğmeye tıklandığında, atama bekleyen tüm başvuruların listelendiği yeni bir sayfa açılır.
- **Jüri Üyeleri Atayın:** Yönetici her bir başvuru için "Jüri Üyelerini Ata" düğmesine tıklayarak belirli sayıda jüri üyesi atayabilir (3 ila 7 arasında). Jüri üyeleri atandıktan sonra başvuru durumu "İşleniyor" olarak güncellenir.

Jüri Üyesi Atama Algoritması

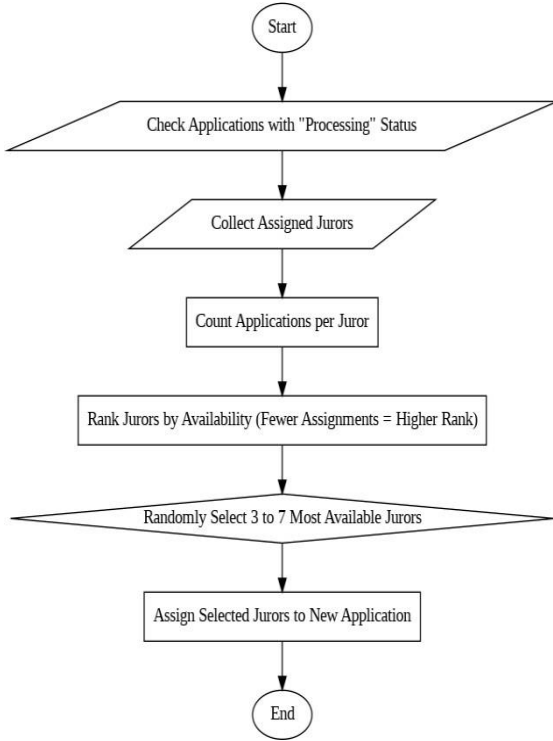
Atama sürecinde adaleti ve verimliliği sağlamak için sistem, jüri üyelerinin uygunluğuna dayalı **rastgele** bir **seçim algoritması** kullanır.

Manuel Seçim Yok: Yöneticinin jüri üyelerini isme veya TC kimliğine göre manuel olarak seçmesine izin vermek yerine, sistem jüri üyelerini müsaitlik durumlarına göre rastgele seçer.

Şeffaflık: Bu yaklaşım, sistem bir "kara kutu" gibi işlediğinden ve kayırmacılığa eğilimli olmadığından, atama sürecinin şeffaf olmasını sağlar.

Zaman Verimliliği: Seçim sürecini otomatikleştiren sistem, özellikle çok sayıda başvurunun ele alındığı durumlarda zamandan tasarruf sağlar. Her başvuruya 3 ila 7 jüri üyesi atanarak hiçbir jüri üyesinin çok fazla başvuru altında ezilmemesi sağlanır.

Algoritma aşağıdaki gibi çalışır:



şekil3. Jüri Atama Algoritması

Müsaitlik Kontrolü: Sistem öncelikle "İşlemde" statüsündeki tüm başvuruları kontrol eder ve bu başvurulara atanan jüri üyelerini toplar.

Jüri Üyelerini Sıralama: Daha sonra tüm jüri üyelerini atandıkları başvuru sayısına göre sıralar. Daha az atanmış başvurusu olan (daha fazla mevcut) jüri üyeleri daha üst sıralarda yer alır.

Rastgele Seçim: Algoritma, bu listeden mevcut başvuruya atanmak üzere en uygun jüri üyelerini (3 ila 7 arasında) seçer.

Bölüm 3: Başvuru Değerlendirmeleri

Yönetici panelinin üçüncü bölümü, jüri üyeleri tarafından sağlanan başvuruların değerlendirmelerini görüntüler. Bu bölüm, yöneticinin jüri üyelerinin her bir başvuru hakkındaki geri bildirimlerini görüntülemesine ve yönetmesine olanak tanır. Bununla ilgili daha fazla ayrıntı daha sonra ele alınacaktır.

D. Değerlendirme Süreci - Jüri Üyeleri

Jüri üyeleri başvurulara atandıktan sonra, sunulan belgeleri incelemek, bilgileri analiz etmek ve bir değerlendirme yapmak için başvuru ayrıntılarına erişebilirler.

Jüri Heyeti

Jüri paneli, durumları da dahil olmak üzere jüri üyesine atanan tüm başvuruları görüntüler:

- **Onaylandı:** Geçmişte onaylanmış başvurular için.
- **Reddedildi:** Geçmişte reddedilen başvurular için.
- **İşleme alınıyor:** Değerlendirmesi devam eden başvurular için.

Jüri üyesi değerlendirmek istediği başvurunun üzerine tıklayabilir. Bu işlem onları **Başvuru Detayları** sayfasına yönlendirir ve burada başvuru sahibinin kişisel bilgilerinin yanı sıra başvuru sürecinde sunulan belgeleri de inceleyebilirler.

Değerlendirme için Düğmeler

Başvuru Ayrıntıları sayfasında, jüri üyesinin bir değerlendirmeyi görüntülemesine veya eklemesine olanak tanıyan iki ana düğme vardır:

- **"Raporu Görüntüle":** Bu düğme, jüri üyesi daha önce bir değerlendirme göndermişse etkinleştirilir. Bu düğmeye tıklandığında jüri üyesi, kararı (onaylandı veya reddedildi), tüm mesajları ve yüklenen rapor dosyasını içeren değerlendirme raporunu görebileceği yeni bir sayfaya yönlendirilir.
- **"Rapor Ekle":** Jüri üyesi henüz bir değerlendirme yapmamışsa, bu düğme etkinleştirilir ve jüri üyesinin kararını bildirmesine olanak tanır. Bu seçenek aktifken "Raporu Görüntüle" düğmesi devre dışı bırakılır.

"Rapor Ekle" düğmesine tıklandığında, jüri üyesine değerlendirmesini girmesi için bir form sunulur:

- **Onay Durumu:** Jüri üyesinin "ONAYLANDI" ve "REDDEDİLDİ" arasında seçim yapmasını sağlayan bir açılır menü.
- **Dosya Yükleme:** Değerlendirme raporu belgesini yüklemek için bir dosya girişi.
- **Metin Alanı:** Jüri üyesinin başvuru hakkında kişisel değerlendirmelerini veya ek yorumlarını sunabileceği bir metin alanı.

Jüri üyesi değerlendirmesini gönderdikten sonra rapor saklanır ve hem jüri üyesi hem de yönetici rapora erişebilir (Salt Okunur).

E. Nihai Karar - Yönetici Paneli

Yönetici panelinin üçüncü bölümü, değerlendirilmek üzere jüri üyelerine atanan tüm başvuruları içerir. Bir jüri üyesi bir başvuru için değerlendirmesini gönderdiğinde, başvurunun durumu jüri üyesinin kararını yansıtacak şekilde güncellenir.

Başvuru Durumu

- Bir jüri üyesi henüz bir değerlendirme göndermemişse, uygulama onlar için tıklanamaz (devre dışı) kalır.
- Bir jüri üyesi değerlendirmesini göndermişse, başvuru durumu düğmesi tıklanabilir hale gelir. Düğmenin rengi değişir:
 - **Kırmızı:** Jüri üyesi başvuruyu reddetmişse.
 - **Yeşil:** Jüri üyesi başvuruyu kabul etmişse.

Başvurunun durum düğmesine tıklamak, yöneticiyi jüri üyesinin kararını, mesajını ve yüklenen rapor dosyasını görüntüleyebilecekleri **Değerlendirme Ayrıntıları** sayfasına yönlendirir.

Daha Fazla Değerlendirme Düğmesi

Bu bölümde, tıklandığında yöneticiyi değerlendirme için atanmış tüm başvuruların listelendiği yeni bir sayfaya yönlendiren bir **"Daha Fazla Değerlendirme"** düğmesi de bulunmaktadır. Bu, yöneticinin aynı anda birden fazla başvuruda kolayca gezinmesine ve bunları yönetmesine olanak tanır.

Nihai Karar

Bir başvuru için jüri üyeleri tarafından tüm değerlendirmeler sunulduktan sonra, yönetici nihai kararı vermekten sorumludur. Bunu yapmak için, yönetici aşağıdakileri içeren bir form kullanır:

- **Onay Durumu:** Yöneticinin **"ONAYLANDI"** veya **"REDDEDİLDİ"** seçeneklerinden birini seçmesine olanak tanıyan bir açılır menü.
- **Metin Alanı:** Yöneticinin başvuruyla ilgili nihai kararını ve yorumlarını girmesi için bir alan.

Nihai karar verildikten sonra kaydedilir ve süreç tamamlanır ve başvuru, yönetici (nihai jüri olarak hareket eden) tarafından sağlanan durumla güncellenir. Başvuru sahibi bilgilendirilir ve başvuru durumunu iş portal üzerinden kontrol edebilir.

5. Bildirimler & Durum Güncellemeleri

Sistem, kullanıcıları başvuru durumları hakkında bilgilendirmek için çeşitli yöntemler içermekte ve ilgili tüm tarafların önemli güncellemelerden derhal haberdar olmasını sağlamaktadır. Bildirimler birden fazla kanal aracılığıyla gönderilir:

Uygulama Yerleşik Bildirimleri:

Bildirimler veritabanında saklanır ve rollerinden bağımsız olarak her kullanıcının portalında görüntülenebilir. Bu, kullanıcıların güncellemelerini doğrudan uygulama içinde görmelerini sağlar.

Twilio (SMS Bildirimleri):

Twilio, SMS bildirimleri için uygulamaya entegre edilmiştir. Ancak, testlerin yapılabilmesi için bir ABD numarasına ihtiyaç duyulması nedeniyle henüz kullanılabilir durumda değildir. Alternatif olarak, test ve operasyonel kullanım için bir Türk numarası satın alınabilir.

Brevo (E-posta Bildirimleri):

Brevo tamamen entegre ve çalışır durumdadır. Kullanıcılara e-posta bildirimleri göndermek için kullanılır ve başvuru süreçleriyle ilgili önemli olaylardan haberdar olmalarını sağlar.

Bildirimler Ne Zaman Gönderilir?

Tüm tarafları bilgilendirmek için başvuru ve değerlendirme sürecinin çeşitli aşamalarında bildirimler tetiklenir. Bunlar şunları içerir:

- Başvuru Gönderimi:

Bir kullanıcı yeni bir başvuru gönderdiğinde, duyuruyu oluşturan yöneticiye yeni başvuru hakkında bilgi veren bir bildirim gönderilir.

- Son Duyuru Tarihine Ulaşıldı:

Bir duyuru son başvuru tarihine ulaştığında, sistem duyuruyu oluşturan yöneticiye, o duyuru için gönderilen toplam başvuru sayısı ile birlikte otomatik bildirim gönderir.

- Jüri Ataması:

Değerlendirme için bir başvuruya jüri üyeleri atandığında, atanan jüri üyelerine yeni sorumluluklarını bildiren bir bildirim gönderilir.

- Jüri Değerlendirme Sunumu:

Bir jüri üyesi değerlendirmesini gönderdikten sonra, duyuruyu oluşturan yöneticiye değerlendirmenin tamamlandığını bildiren bir bildirim gönderilir.

- Nihai Karar:

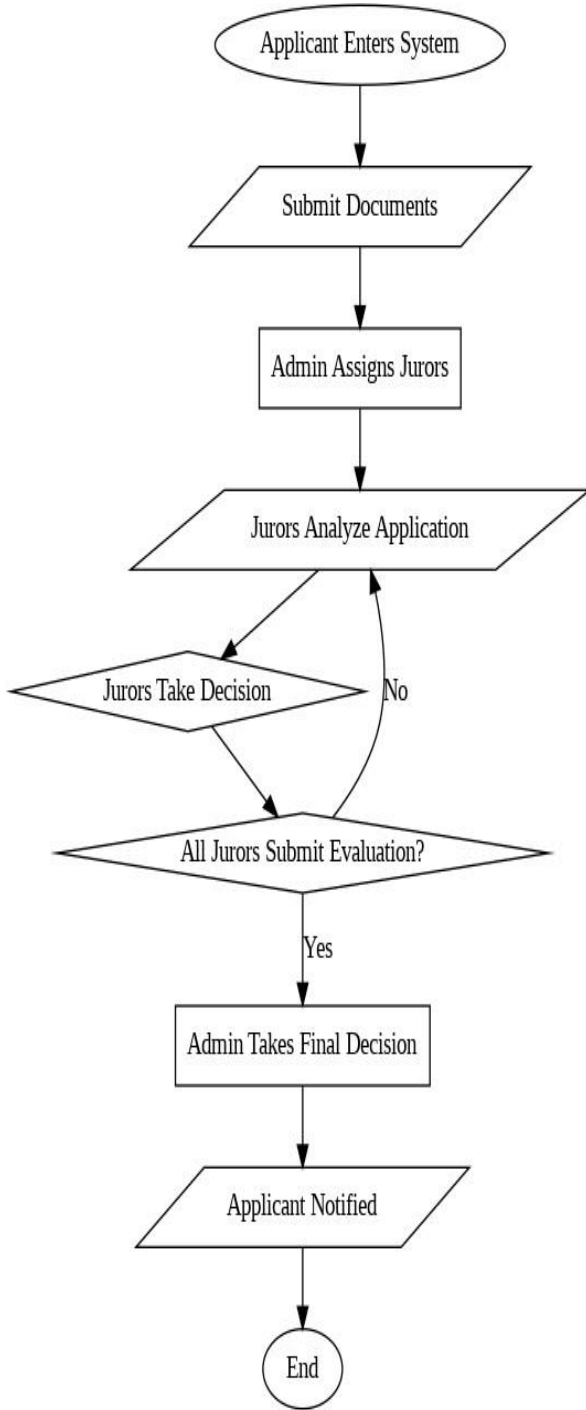
Nihai karar yönetici tarafından verildikten sonra, başvuru sahibine başvurusunun sonucunu bildiren bir bildirim gönderilir.

Bu bildirimler, başvuru süreci boyunca şeffaflık ve zamanında iletişim sağlar.

Nihai Karar ve İşe Alım

Onaylanan adaylar bilgilendirilir ve bilgileri daha sonraki işlemler için sistemde saklanır.

Yani başvurunun sunulmasından sonuca kadar sistem şu şekilde özetlenebilir:



Şekil 4. Uygulama Süreci

Yönetici Paneli

Yönetici sistemin belkemiği olarak görev yapar. Yöneticiler ve Jüri Üyelerinin aksine, Yönetici başvuru sürecine müdahale etmez. Ancak tüm kullanıcılar -Yöneticiler, Başvuru Sahipleri ve Jüri Üyeleri- Yönetici tarafından önceden tanımlanan kriterlere dayanır. Bu da Yöneticiyi sistemin temel bir bileşeni haline getirir.

Yönetici Paneli, her biri başvuru sürecinin yapılandırılmasında önemli bir rol oynayan altı bölüme ayrılmıştır.

i. Kriterleri Düzenle

Bu bölüm, Yöneticinin **KOÜ Atama Yönetmeliği** temelinde temel değerlendirme kriterlerini tanımlamasına olanak tanıyan 3'ü 1 arada bir form sunmaktadır. Bu form şunları içerir:

- Tablo 3 (Faaliyet Kategorileri ve Faaliyetler):

Tek bir form, tüm bir kategoriye sonraki tüm faaliyetleriyle birlikte bir kerede eklemek için kullanılır.

Örnek: L (GÜZEL SANATLAR FAALİYETLERİ)

kategorisi 105 faaliyet içerir ve bunların tümü 105 ayrı çağrı yerine tek bir arka uç talebiyle eklenebilir. Bu, verimlilik sağlar ve sunucu yükünü azaltır.

- Tablo 1 (Asgari Faaliyetler):

Her bir fakülte ve pozisyon için gerekli asgari faaliyetleri tanımlar (örneğin, Yardımcı Doçent, Doçent, vb.).

- Tablo 2 (Minimum ve Maksimum Puanlar):

Fakülte ve pozisyona göre gerekli minimum ve maksimum puanları belirtir.

ii. Faaliyetler (tablo 3 - görünüm)

Sistemde o anda mevcut olan tüm faaliyetleri ilişkili puanlarıyla birlikte görüntüler.

Yönetici hariç herkes için salt okunur erişim.

Yönetici şunları yeteneğine sahiptir:

- Mevcut bir faaliyeti değiştirin.
- Bir etkinliği silin.
- Tüm kategorileri değiştirin/silin.

iii. Minimum Faaliyetler

Tablo 1'i görüntüler: Fakülte ve pozisyona göre her kategori için gerekli minimum faaliyetler.

Kriterleri Düzenle formu aracılığıyla girilen veriler şu şekilde yapılandırılır:

- Aralıklar (örn. A1-A4).
- Belirli kriterler (örneğin, "Ana Yazar").

Yönetici dışındaki tüm kullanıcılar için salt okunur.

Yönetici şunları yapabilir:

- Mevcut bir kriteri değiştirin.
- Bir kriteri silin.

iv. Minimum ve Maksimum Puanlar

Tablo 2'yi gösterir:

- Kategori başına gereken minimum puanlar.
- Kategori başına izin verilen maksimum puan. Yönetici hariç tüm kullanıcılar için salt okunur. Yönetici şunları yapabilir:
- Mevcut bir nokta aralığını değiştirin.

- Bir nokta aralığını silin.

v. Vaka ve Katsayı (Tablo 4)

KOÜ Atama Yönetmeliği'ne göre bazı kategoriler (A, B, C, G) ya

- Özel bir durum ("DURUM") veya
- Özel bir durum yoksa bir katsayı ("ÇARPAN").

Bu bölüm aşağıdakilere karşılık gelmektedir: Tablo 4 (Katsayılar Tablosu).

"Özel Durumlar" bölümü (KOÜ Atama Yönetmeliği Sayfa 22).

Uygulamalarda otomatik nokta hesaplaması için kullanılır.

Bu bölümdeki verileri yalnızca Yönetici değiştirebilir, ekleyebilir veya silebilir.

vi. Yönetici Formu (Dinamik Başvuru Formu)

Sistemin en kritik bölümlerinden biri.

Başvuru Sahiplerinin başvuru sürecinde doldurduğu dinamik formu temsil eder.

Tüm form yapısı veritabanında saklanır.

Formları neden dinamik olarak saklayalım?

KOÜ Atama Yönetmeliği'ndeki yeni kategoriler veya değişiklikler çekirdek sistemi değiştirmeden barındırılabilirdiğinden ölçeklenebilirlik sağlar.

Sadece Müdür yapabilir:

- Mevcut bir formu değiştirin.
- Bir formun tamamını silin.
- Yeni bir form ekleyin.
- Bir formdaki belirli alanları değiştirin/silin.

Her bir form KOÜ Atama Yönetmeliği Tablo 3'te yer alan bir kategoriye karşılık gelmekte, böylece yapılandırılmış ve tutarlı bir değerlendirme süreci sağlanmaktadır.

Yönetici Paneli, başvuru sürecini yönlendiren kuralları, kriterleri ve puanlama sistemini tanımlayan temel idari arayüz olarak hizmet vermektedir. Yönetici (yöneten) veya Jüri Üyelerinin (değerlendiren) aksine, Yöneticinin rolü tutarlılık, adalet ve KOÜ Atama Yönetmeliği'ne uygunluk sağlayan değerlendirme çerçevesini yapılandırmaktır.

Bu yapılandırılmış yaklaşım, esneklik ve ölçeklenebilirlik sağlayarak sistemin önemli değişiklikler olmadan gelecekteki değişikliklere uyum sağlamasına olanak tanır.

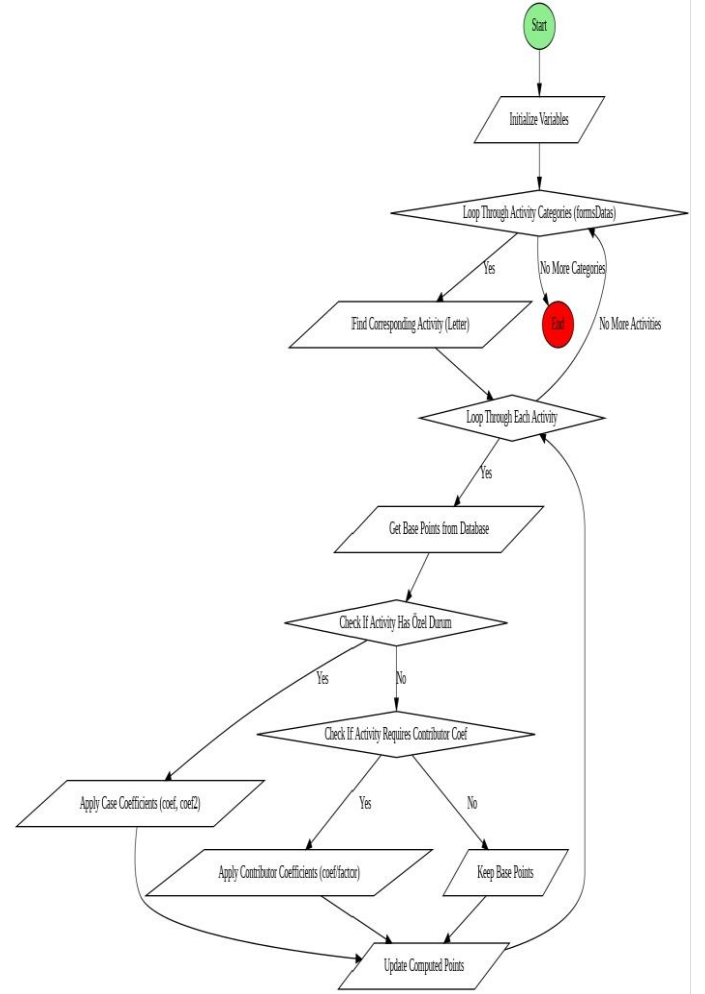
6. Nokta Hesaplama Algoritması ve Mekanizması

Nokta Hesaplama Fonksiyonu (computesPoints)

computesPoints işlevi, başvuru sahibi tarafından sunulan her bir faaliyet için toplam puanların hesaplanmasından sorumludur.

Hesaplama, KOÜ Atama Yönetmeliği'nde tanımlanan kuralları izler ve Yönetici tarafından Vaka Katsayısı Sayfasına girilen önceden tanımlanmış katsayıları ve vakaları kullanır.

Algoritma Akış Şeması



şekil5. Nokta Hesaplama Algoritması

Fonksiyon, faaliyet için hesaplanan son noktaları saklamak için `updatePoints(letter, number, round(activityPoints))` çağrısında bulunur.

Hesaplama Sürecinde Yöneticinin Rolü

- Katsayıların Tanımlanması ve Özel Durumlar:
- Yönetici, Case Coef Sayfasında **Özel Durumlar** için katsayı değerlerini (coef, coef2) önceden tanımlar.
 - Özel bir durum varsa, hesaplamada coef ve coef2 kullanılır.
 - Özel durum yoksa, bunun yerine katkıda bulunanların sayısına dayalı bir katsayı kullanılır.
- Adil ve Şeffaf Değerlendirmenin Sağlanması: Önceden tanımlanmış kuralların kullanılması, her başvuru sahibinin tutarlı bir şekilde değerlendirilmesini sağlar.

Tüm kriterler veritabanında saklanır ve puan hesaplama işlemi sırasında dinamik olarak uygulanır.

III. SİSTEM BİLEŞENLERİ

Bu bölüm, Akademik Personel İşe Alım Sisteminin ön uç, arka uç, veritabanı, kimlik doğrulama, dosya depolama, bildirimler ve harici API entegrasyonlarını kapsayan temel bileşenlerinin ayrıntılı bir dökümünü sunmaktadır.

1. Ön Uç (React.js & Bootstrap)

Sistemin ön ucu, dinamik ve duyarlı kullanıcı arayüzleri sağlayan modern bir JavaScript kütüphanesi olan **React.js** kullanılarak oluşturulmuştur. **Bootstrap**, profesyonel ve mobil uyumlu bir tasarım sağlamak için şekillendirme için kullanılır.

Temel Özellikler:

- **Kullanıcı Kimlik Doğrulama:** Rol tabanlı erişim ile oturum açma ve kayıt.
- **Gösterge Paneli:** Her kullanıcı için kişiselleştirilmiş arayüzler: başvuru sahipleri, yöneticiler, jüri ve yönetici üyeler.
- **Başvuru Yönetimi:** Adaylar başvurularını gönderebilir ve takip edebilirler.
- **Gerçek Zamanlı Güncellemeler:** API yanıtlarını kullanarak uygulama durumu güncellemeleri.

Kullanılan Ana Teknolojiler:

- **React.js:** Verimli kullanıcı arayüzü geliştirme için bileşen tabanlı mimari.
- **React-native-dom Yönlendirici:** Farklı görünüm arasında gezinmeyi yönetir.
- **Bootstrap:** Duyarlı ve temiz bir kullanıcı arayüzü sağlar.
- **Fetch API:** Üçüncü taraf kütüphaneler olmadan arka uca yapılan API çağrılarını yönetir.

2. Arka Uç (Node.js & Express.js)

Arka uç, **Express.js** çerçevesi ile **Node.js** kullanılarak geliştirilmiştir. İş mantığını ele alan, istekleri işleyen ve veritabanıyla etkileşime giren **merkezi merkez** olarak hizmet vermektedir.

Temel Özellikler:

- **RESTful API Geliştirme:** Ön uç etkileşimi için uç noktaları ortaya çıkarır.
- **Rol Tabanlı Erişim Kontrolü:** Farklı kullanıcı izinlerini yönetir (Başvuru Sahibi, Yönetici, Jüri, Yönetici).
- **Doğrulama ve Hata İşleme:** Veri bütünlüğünü ve güvenliğini sağlar. Bir hata oluştuğunda net bir mesaj görüntüler.

Kullanılan Ana Teknolojiler:

- **Node.js:** Asenkron ve verimli arka uç işleme.
- **Express.js:** API'ler oluşturmak için hafif çerçeve.
- **Mongoose:** MongoDB entegrasyonu için ORM.
- **Multer:** Dosya yönetimi.
- **Amazon S3:** Dosya depolama

3. Orta Arka Uç Katmanı : React Bağlıları

Bu projede, kullanıcı oturumlarını, rol tabanlı erişimi ve API (arka uç) iletişimini verimli bir şekilde yönetmek için ön uç ve arka uç arasında bir ara katman tanıtıldı. Bu katman, **React Contexts** kullanılarak uygulanmış, global durum yönetimi ve **fetch API** çağrıları aracılığıyla arka uç noktalarıyla etkileşim için merkezi mantık sağlamıştır. Aşağıdaki beş ana bağlam oluşturulmuştur:

- **UserContext:** JWT belirteci işleme, kişisel veriler ve atanan roller dahil olmak üzere kimliği doğrulanmış kullanıcının oturumunu yönetir. Ayrıca burada, uygulamanın girişinde, kaydında her kullanıcı rolü için minimum veri yüklenir.
- **ManagerContext:** Yönetici ile ilgili tüm işlemler için (daha önce bahsedildiği gibi)
- **AdminContext:** Yönetici ile ilgili tüm işlemler için (daha önce konuşulduğu gibi)
- **JuryContext:** Jüri ile ilgili tüm işlemler için (daha önce konuşulduğu gibi).
- **NotificationContext:** Tüm sistem bildirimlerini (başarı, hata, bilgi) işleyerek uygulama boyunca tutarlı ve kullanıcı dostu geri bildirim sağlar.

Ön uca uygulanmasına rağmen, bu bağlamlar "orta arka uç-uç"^[1] katmanı olarak hizmet eder, çünkü tamamen sunumsal bileşenlerden temiz bir ayrımı korurken arka uca doğrudan iletişim kuran mantığı merkezileştirirler.

Context API'nin Faydaları

React Context API kullanımı çeşitli avantajlar sağlar:

- **Yerel ReactJS API:** kurulum gerektirmez
- **Küresel Durum Paylaşımı:** Kullanıcı rolleri veya oturum açma belirteçleri gibi verilere, prop sondajı olmadan tüm bileşenler erişilebilir.
- **Modüler Mantık:** API işlevleri ve rol tabanlı mantık, kod modülerliğini ve yeniden kullanılabilirliği teşvik ederek her bağlam içinde kapsüllenir.
- **Daha Temiz Bileşen Yapısı:** İş mantığı kendi bağlamlarında bulunurken bileşenler kullanıcı arayüzü endişelerine odaklanmaya devam eder.
- **Geliştirilmiş Sürdürülebilirlik:** Kullanıcı arayüzü, mantık ve veri getirme arasındaki sorumlulukların ayrılması, hata ayıklamayı ve gelecekteki geliştirmeleri basitleştirir.

Context API vs Redux

Context API projenin ölçeği ve ihtiyaçları için yeterli olsa da, gelecekte daha gelişmiş senaryolar için **Redux** ile entegrasyon düşünülebilir:

- **Karmaşık Durum Bağımlılıkları:** Redux'un redüktör tabanlı modeli, derinlemesine iç içe geçmiş veya birbirine bağlı durum güncellemelerini yönetmek için daha uygundur.
- **Zaman Yolculuğu Hata Ayıklama:** Redux DevTools, durum değişikliklerinin hassas bir şekilde izlenmesini sağlayarak karmaşık iş akışlarında hata ayıklamaya yardımcı olur.
- **Ara Yazılım Desteği:** Redux ara yazılımı (ör. redux-thunk veya redux-saga) aşağıdakiler üzerinde güçlü kontrol sağlar

arka uç etkileşim katmanını geliştirebilecek asenkron mantık.

4. Veritabanı (MongoDB)

Sistem, uygulama ile ilgili verileri esnek JSON benzeri bir formatta depolamak için bir NoSQL veritabanı olan **MongoDB**'yi kullanır.

Veritabanı Yapısı:

- **Modeller** : Kullanıcı, Fakülte, Faaliyet, Duyuru, Başvuru, Vaka, Coef, Bölüm, Değerlendirme, FakülteGrubu, Form, MinAktivite, MinPoint, Bildirim.
- **Grafik Diyagramı** : Veritabanı diyagramı bir sonraki bölümde verilmiştir.

Neden MongoDB?

- **NoSQL** : mongoDB noSQL yapısı ile, JSON ile benzerliği nedeniyle veri işlemek kolaydır.
- **Şema esnekliği** dinamik veri depolamaya olanak tanır.
- **Yüksek ölçeklenebilirlik**, artan kullanıcı verilerini destekler.
- Gerçek zamanlı uygulama takibi için **hızlı sorgulama**.

5. Kimlik Doğrulama & Güvenlik (JWT - JSON Web Token)

Kimlik doğrulama, güvenli kullanıcı erişimi ve rol yönetimi sağlayan **JWT** kullanılarak gerçekleştirilir.

Kimlik Doğrulama Süreci:

Kullanıcılar e-posta ve şifre ile kayıt olurlar.

Sistem kimlik bilgilerini doğrular ve bir **JWT belirteci** oluşturur.

Belirteç, güvenli API erişimi için her istekle birlikte gönderilir.

Ara yazılım, rollere göre erişime izin vermek için belirteçleri doğrular.

Güvenlik Önlemleri:

bcrypt kullanarak **parola karma**.

Yetkisiz erişimi önlemek için **belirteç süresinin dolması**.

Farklı kullanıcı türleri için **rol tabanlı izinler**.

6. Dosya Depolama (Amazon S3)

Sistem, güvenli ve ölçeklenebilir dosya depolama için **AWS S3**'ü entegre eder.

Belge İşleme:

Başvuru sahipleri gerekli belgeleri (CV'ler, sertifikalar, vb.) yükler.

Dosyalar **Amazon S3 kovalarında** güvenli bir şekilde saklanır. Kontrollü erişim için URL'ler oluşturulur.

Ayrıca tablo 5 pdf oluşturulan temel don kullanıcı girdileri diğer dosyalarla birlikte kovaya gönderilir.

Amazon S3'ün Faydaları:

Ölçeklenebilirlik: Büyük belge depolamayı verimli bir şekilde yönetir.

Güvenlik: Şifrelenmiş dosya depolama sağlar.

Erişilebilirlik: Dosyalara hızlı erişim ve kontrollü erişim.

7. API Entegrasyonları (e-Devlet & Nüfus Müdürlüğü)

Sistem, başvuru sahibi bilgi ve belgelerini doğrulamak için **e-Devlet** ve **Nüfus Müdürlüğü API'leri** ile entegre olur.

Entegrasyon Adımları:

Sistem, kimlik doğrulaması için kullanıcı bilgilerini **e-Devlet API**'sine gönderir.

API doğrulama sonuçları ile yanıt verir.

Onaylanan başvuru sahipleri bir sonraki işe alım aşamasına (kayıt) geçerler.

API Entegrasyonu Neden Önemlidir?

Gerçeklik: Sadece meşru adayların başvurmasını sağlar.

Otomasyon: Manuel belge kontrollerini azaltır.

Verimlilik: Doğrulama sürecini hızlandırır.

IV. FONKSİYONLAR & KULLANICI ROLLERİ

Bu bölümde Akademik Personel İşe Alım Sisteminin **temel işlevleri** açıklanmakta ve platformda yer alan farklı **kullanıcı rolleri** detaylandırılmaktadır. Sistem, her kullanıcının belirli eylemleri gerçekleştirmek için gerekli izinlere sahip olmasını sağlayacak şekilde **rol tabanlı erişim kontrolü** sağlamak üzere tasarlanmıştır.

A. Temel İşlevler

Sistem aşağıdaki temel işlevleri sağlar:

1. Kullanıcı Kimlik Doğrulama ve Yetkilendirme

Kullanıcılar **JWT tabanlı kimlik doğrulama** kullanarak kaydolabilir ve oturum açabilir.

Rol tabanlı erişim, farklı kullanıcıların farklı izinlere sahip olmasını sağlar.

2. Başvuru Gönderimi ve Takibi

Başvuru sahipleri başvurularını online olarak **doldurup gönderebilirler**.

'de depolanan gerekli belgeleri yükleyebilirler.

Amazon S3.

Bir gösterge paneli, başvuru sahiplerinin başvurularının **durumunu takip** etmelerini sağlar.

3. Belge Doğrulama ve Geçerleme

Sistem, kullanıcı bilgilerini **e-Devlet & Nüfus Müdürlüğü API**'si üzerinden doğrular.

Doğrulama süreci yalnızca geçerli başvuruların işleme alınmasını sağlar.

4. Jüri Üyeleri Tarafından İnceleme ve Değerlendirme

Jüri üyeleri, değerlendirme için gönderilen başvurulara erişebilir.

Onaylayabilir, reddedebilir veya ek belgeler talep edebilirler.

İnceleme süreci takip edilir ve değerlendirmeler veri tabanında saklanır.

5. Bildirim Sistemi (Twilio API)

SMS ve e-posta bildirimleri başvuru sahiplerini başvuru durum güncellemeleri hakkında bilgilendirir.

Jüri üyeleri, dikkatlerini gerektiren başvurular olduğunda uyarı alırlar.

6. Rol Tabanlı Gösterge Tabloları

Her kullanıcı kendi rolüne göre uyarlanmış bir gösterge tablosu görür:

Başvuru Sahipleri: Başvuru ilerlemesini takip edin ve bildirimler alın.

Yöneticiler: Kullanıcı hesaplarını yönetir, uygulamaları denetler ve anlaşmazlıkları ele alır.

Jüri Üyeleri: Başvuruları inceleyin ve değerlendirmeleri gönderin.

Yöneticiler: Tüm işe alım sürecini denetleyin ve raporlar oluşturun.

B. Kullanıcı Roller ve İzinleri

Sistem, her biri belirli izinlere sahip **dört ana kullanıcı rolü** tanımlar:

Rol	İzinler
Başvuru sahibi	- Kayıt olun ve giriş yapın - Duyuru oluşturun ve güncelleyin - Başvuruları gönderin - Belgeleri yükleyin - Başvuru durumunu takip edin - Bildirim alma
Yönetici	- Kullanıcı hesaplarını yönetin - Jüri üyelerini atayın - Başvuru işlemlerini denetleyin - Son kararı verin
Jüri Üyesi	- Başvuruları inceleyin ve değerlendirin - Bir başvuruyu onaylayın veya reddedin - Başvuru sahipleri için geri bildirim bırakın
Müdür	- Raporlar ve analizler oluşturma - Sistem genelindeki ayarları yönetme

Tablo 2. Kullanıcı Roller ve İzinleri

V. UYGULAMA DETAYLARI

Bu bölümde **Akademik Personel İşe Alım Sisteminin** teknik uygulaması özetlenmekte ve **ön uç, arka uç, veritabanı, kimlik doğrulama, depolama, bildirimler ve API entegrasyonlarına** odaklanılmaktadır.

1. Ön Uç Uygulaması

Ön uç **React.js** ile oluşturulmuştur ve stil için **Bootstrap** kullanılmaktadır. Modülerlik ve yeniden kullanılabilirlik sağlayan **bileşen tabanlı bir mimari** izler.

Temel Uygulama Detayları:

Yönlendirme: **React-native-dom Router** kullanılarak uygulandı Navigasyon için.

Kullanıcı uygulaması için **Dinamik MultiStep formu**

Durum Yönetimi: React'in **useState** ve **useEffect** kancaları durum değişikliklerini işlemek için kullanılır.

API Çağrıları: **Fetch API** arka uç ile iletişim için kullanılır. Tüm API çağrıları ContextAPI (UserContext, ManagerContext, AdminContext, JuryContext, NotificationContext) kullanılarak Context içinde merkezileştirilir.

Fetch - UserContext.js kullanarak Örnek API Çağrısı:

```
fetch('https://{api_address}/api/datas/applications/view', {
  method: 'GET',
  headers: {
    'Yetkilendirme': `Bearer ${token}`,
    'Content-Type': 'application/json'
  }
})
.then(yanıt=> yanıt.json())
.then(veri=> console.log(veri))
.catch(hata=> console.error('Hata:', hata));
```

Kod 1. Getirme API çağrısı örneği

2. Arka Uç Uygulaması

Arka uç, istekleri işlemek, iş mantığını işlemek ve veritabanını yönetmek için **Express.js** ile **Node.js** kullanılarak geliştirilmiştir.

Temel Uygulama Detayları:

Yönlendirme: Express.js, RESTful API rotalarını tanımlamak için kullanılır.

Orta katman yazılımı: JSON Web Token (**JWT**) kimlik doğrulaması, Amazon Web S3 ile multer uygulanmaktadır.

Veritabanı İletişimi: **Mongoose**, MongoDB etkileşimleri için kullanılır.

Harici Bildirim Servisleri : Twilio, Brevo

Node JS Yardımcı Programları : Bir duyurunun süresi dolduğunda Yöneticiye otomatik olarak bir bildirim göndermek için mısır inethrated

Örnek index.js Rotası:

```
const express= require('express'); const
router= express.Router();
const auth= require('../middlewares/auth');

const announcementRoutes=
require('./announcementRoutes');
const applicationRoutes= require('./applicationRoutes');
/*diğer rotalar*/
```

```
module.exports= router;
```

Kod 2. Bir routes/index.js dosyası örneği

Örnek applicationRoutes.js Rota:

```
const express= require("express");
const router= express.Router(); const
applicationController =
require("../controllers/applicationController"); const {
applicationsDocUpload }=
require("../middlewares/awsMiddleware");

router.get("/get/all", applicationController.getApplications);
router.get("/user/get/:userId",
applicationController.getUserApplications);

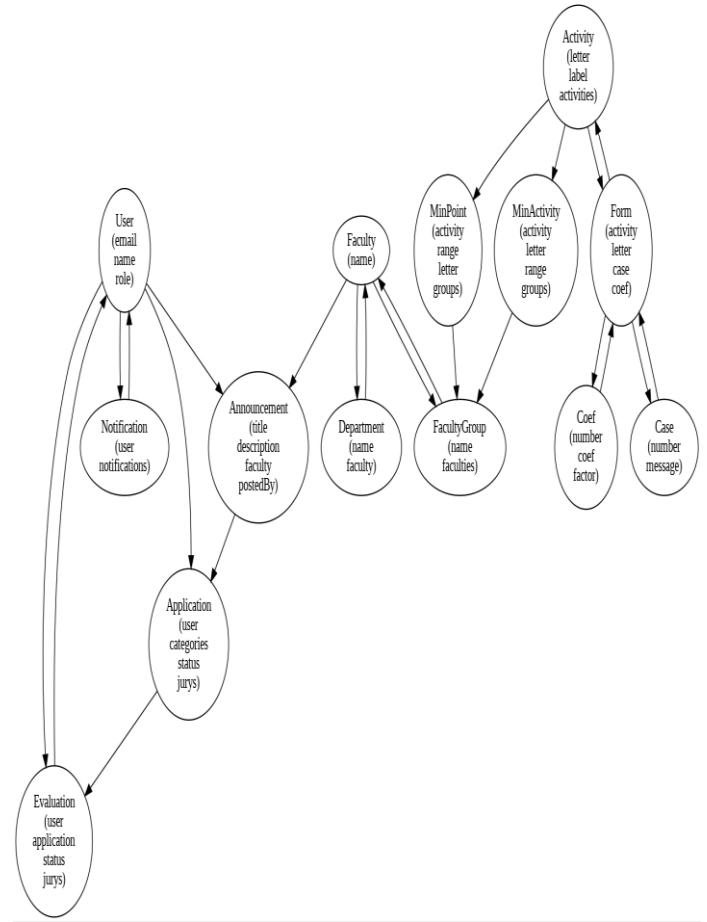
module.exports= router;
```

Kod 3. applicationRoutes.js örneği

3. Veritabanı Uygulaması (MongoDB)

Sistem, kullanıcı ve uygulama verilerini depolamak için bir NoSQL veritabanı olan **MongoDB**'yi kullanır.

Veritabanı Diyagramı:



şekil6. Veritabanı ER Diyagramı

Veritabanı Şema Örnekleri:

Kullanıcı Şeması:

```
const mongoose= require('mongoose'); const
Schema= mongoose.Schema;
const uniqueValidator= require('mongoose-unique-validator'); const
db= mongoose.connection.useDb("kouap");
const userSchema= new mongoose.Schema({ name: {
type: String, required: true, trim: true },
surname: { type: String, required: false, trim: true }, tcID: {
type: String, required: true, trim: true }, birthDate: { type:
Date, required: true, trim: true },
username: { type: String, required: false, trim: true },

e-posta: { type: String, required: true, unique: true, lowercase: true,
trim: true },
phoneNumber: { type: String, required: true, trim: true }, address: {
type: String, required: true, },
password: { type: String, required: true },
role: { type: String, enum: ['user', 'admin', 'jury', 'manager', 'dev'],
default: 'user' },
cv : { type: String, required: function(){ return
this.role== 'user' } },
```

```
signature : { type: String, required: function(){ return this.role==
'user' } },

userSchema.plugin(uniqueValidator); module.exports=

db.model("User", userSchema);
```

Kod 4. Mongoose/MongoDB Kullanıcı Modeli Tanım

4. Arka Uçtan Görülen Kimlik Doğrulama (JWT - JSON Web Token)

Kullanıcı kimlik doğrulaması, kullanıcı oturumlarını güvenli bir şekilde doğrulayan ve koruyan **JWT** kullanılarak yönetilir.

Kimlik Doğrulama Akışı:

Kullanıcı oturum açar ve kimlik bilgilerini sağlar.

Sistem kimlik bilgilerini doğrular ve bir **JWT belirteci** oluşturur.

Belirteç, güvenli API erişimi için kullanılır.

Ara katman yazılımı, yalnızca yetkili kullanıcıların belirli uç noktalara erişebilmesini sağlar.

Örnek Oturum Açma Uygulaması:

```
exports.loginUser= async (req, res, next)=> { try
{
validePassword= false olsun;
const { tcID, password}= req.body

const user= await User.findOne({ tcID: tcID}) if

(!user)
{
console.log('auth/user-not-found')
return res.status(401).json({ error: 'auth/user-not-
bulundu'
});
}

if(!isBcryptHash(password))
{
validePassword= await
bcrypt.compare(password.toString(), user.password)
}
başka
{
validePassword= password.toString()===
user.password.toString()
}
```

```
if (!validePassword) {
console.log('auth/incorrect-password')
return res.status(401).json({ error: 'auth/incorrect-
password' });
}
başka
{
const token= generateToken(user._id);
res.status(200).json({message: "success", data:{
token:token, user : user }});
}

}
catch(hata)
{
console.log(hata)
res.status(500).json({ hata });
}
};
```

Kod 5. Kullanıcı için denetleyici: LoginUser Yöntemi

5. Dosya Depolama (Amazon S3)

Sistem, **güvenli dosya yükleme** ve belge depolama için **Amazon S3** kullanıyor.

Dosya Yükleme İş Akışı:

Kullanıcı bir belge yükler. Dosya bir

S3 kovasında saklanır.

Sistem erişim için güvenli bir **URL** oluşturur.

Multer kullanarak Örnek S3 Dosya Yükleme:

```
const multer= require("multer");
const { S3Client }= require("@aws-sdk/client-s3"); const
multerS3= require("multer-s3");

const s3Client= new S3Client({ region:
process.env.AWS_REGION,
credentials: { accessKeyId:
process.env.AWS_ACCESS_KEY_ID, secretAccessKey:
process.env.AWS_SECRET_ACCESS_KEY },
});

const MIME_TYPES= {
"image/jpg": "jpg", "image/jpeg": "jpeg", "image/png": "png",
"application/pdf": "pdf",
```

```

"application/msword": "doc",
"application/vnd.openxmlformats-officedocument.wordprocessingml.document": "docx"
});

const fileFilter= (req, file, cb) => cb(MIME_TYPES[file.mimetype]
? null : new Error("Type de fichier non autorisé"),
!!MIME_TYPES[file.mimetype]);

const multerS3Config= (folder)=> multerS3({ s3:
  s3Client, bucket:
process.env.AWS_S3_BUCKET_NAME, metadata: (req, file, cb) =>
cb(null, { fieldName: file.fieldname },
  key: (req, file, cb)=> cb(null, `${folder}/${Date.now()}-${
file.originalname} `)
});

module.exports= {
  cvUpload: multer({ storage: multerS3Config("cv"), fileFilter
}),
  evaluationDocsUpload: multer({ storage:
multerS3Config("evaluationReports"), fileFilter }),
  applicationsDocUpload: multer({ storage:
multerS3Config("applications"), fileFilter }),
};

```

Kod 6. Multer ve AWS kullanarak ara katman yazılımı

6. Bildirim Sistemi (Twilio API)

Twilio, SMS ve e-posta bildirimleri göndermek için entegre edilmiştir başvuru durumu güncellemeleriyle ilgili olarak.

Örnek Twilio Entegrasyonu:

```

require('dotenv').config({ path: '../shared/.env' }); const
mongoose= require('mongoose');
const Notification= require('../models/notificationModel'); const
User= require('../models/userModel');

const twilio= require('twilio'); const

client= new
twilio(process.env.TWILIO_ACCOUNT_SID,
process.env.TWILIO_AUTH_TOKEN);

exports.sendSms= async (to, message)=> { try {
  const response= await client.messages.create({
    body: message,
    from: process.env.TWILIO_PHONE_NUMBER, to:
    to,
  });
};

```

```

console.log("SMS envoyé avec succès :", response.sid); return
response;
} catch (hata) {
  console.error("Erreur lors de l'envoi du SMS :", error.message);
  hata veriyor;
}
};

```

Kod 7. Twilio Entegrasyonu

7. API Entegrasyonları (e-Devlet & Nüfus Müdürlüğü)

Sistem, başvuru sahibi doğrulaması için **e-Devlet & Nüfus Müdürlüğü API'leri** ile entegre çalışmaktadır.

Entegrasyon Süreci:

Sistem, başvuru sahibi verilerini **e-Devlet API**'sine gönderir.

API verileri doğrular ve doğrulama sonuçları ile yanıt verir.

Sistem başvuru durumunu buna göre günceller.

Örnek API İsteği:

```

const generateSoapRequest= (tcID, name, surname, birthYear) =>
`
<?xml version="1.0" encoding="utf-8"?>
<soap12:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
"http://www.w3.org/2003/05/soap-envelope"xmlns:soap12=>
  <soap12:Gövde>
    <TCKimlikNoDogrula xmlns="http://tckimlik.nvi.gov.tr/WS">
      <TCKimlikNo>${BigInt(tcID)}</TCKimlikNo>
      <Ad>${name}</Ad>
      <Soyad>${soyadı}</Soyad>
      <DogumYili>${doğumYili}</DogumYili>
    </TCKimlikNoDogrula>
  </soap12:Body>
</soap12:Envelope>`;

exports.controlUser= async (req, res)=> {
  const { tcID, name, surname, birthYear }= req.body; const
  userID= await User.findOne({ tcID });

  if (userID) return res.status(401).json({ error: 'auth/user-
already-exists' });

  const soapRequest= generateSoapRequest(tcID, name, surname,
  birthYear);

  dene {

```

```

const response= await
fetch('https://tckimlik.nvi.gov.tr/Service/KPSPublic.asmx', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/soap+xml; charset=utf-8',
    'SOAPAction':
'http://tckimlik.nvi.gov.tr/WS/TCKimlikNoDogrula',
  },
  body: soapRequest,
});

if (!response.ok) return res.status(response.status).json({ error:
'Failed to call SOAP API' });

const responseText= await response.text(); const
{"soap:Envelope": { "soap:Body": [{
TCKimlikNoDogrulaResponse: [{ TCKimlikNoDogrulaResult:
[sonuç] ] } ] } } = await xml2js.parseStringPromise(responseText);

if (result=== 'true') return res.status(200).json({ message: 'User
information is correct' });
return res.status(400).json({ error: 'User information is incorrect'
});

} catch (hata) {
console.error('SOAP API çağrılırken hata oluştu:', hata);
return res.status(500).json({ error: 'Failed to call SOAP API'
});
}
};

```

Kod 8. E-Devlet API'sine SOAP Çağrısı

Twilio Entegrasyonu ve Coğrafi Sınırlamalar

Twilio'nun ücretsiz sürümü yalnızca ABD'de bulunan telefon numaralarına mesaj izin veriyor. Bu coğrafi kısıtlama nedeniyle, e-posta bildirimleri için geçici olarak **Brevo**'ya (eski adıyla Sendinblue) geçmek zorunda kaldım. Neyse ki, sistem zaten uyarılar için veritabanını kullanan kendi bildirim sistemine sahip.

Bu sınırlamaya rağmen Twilio sisteme iyi entegre edilmiş ve sorunsuz çalışıyor. Karşılaşılan tek sorun, Twilio'nun ücretsiz sürümünde kısıtlı Türk telefon numarası satın alınamamasıydı.

VI. LEXICON

[1] Orta Arka Uç:

Bu projede ön uç ile arka uç arasındaki bir katmanı ifade etmek için kullanılan özel bir mimari terim. Bu "orta arka uç" React Contexts kullanılarak uygulanmıştır ve

Global uygulama durumunu yönetmekten, arka uca yapılan API çağrılarını işlemekten ve kullanıcı rollerini ve mantık akışlarını kontrol etmekten sorumludur. Teknik olarak ön ucun bir parçası olmasına rağmen, bu katman arka uçla ilgili işlemleri (veri alma, yanıtları işleme ve kimlik doğrulama belirteçlerini yönetme gibi) gerçekleştirir, böylece UI bileşenleri ve arka uç hizmetleri arasında hibrit bir arayüz görevi görür. Kod organizasyonunu ve modülerliği geliştirir ve sunum ile mantık katmanları arasında temiz bir ayrım sağlar.

VII. Sonuç ve Gelecek Çalışmalar

Bu bölümde projenin başarıları özetlenmekte, güçlü yönleri vurgulanmakta ve gelecekteki iyileştirmeler için potansiyel alanlar ana hatlarıyla belirtilmektedir.

1. Proje Başarılarının Özeti

Proje, adayların başvuru yapmasına, bildirim almasına ve bir jüri tarafından değerlendirilmesine olanak tanıyan bir akademik başvuru sistemini başarıyla hayata geçirdi. e-Devlet ve Nüfus Müdürlüğü API'lerinin entegrasyonu aday doğrulamayı kolaylaştırırken AWS S3 güvenilir belge depolamayı sağladı. ||

2. İyileştirme Alanları ve Gelecekteki Geliştirmeler

Potansiyel İyileştirmeler: Gelecekteki geliştirmeler, jürilerin adayların başvurularını daha verimli bir şekilde değerlendirmelerine yardımcı olmak için makine öğrenimi modellerinin eklenmesini . Ayrıca, bir mobil uygulama sürümü hareket halindeki kullanıcılar için erişilebilirliği artırılabilir.

Uzun Vadeli Vizyon: Uzun vadede, sistem birden fazla üniversiteyi destekleyecek şekilde ölçeklendirilebilir ve adayların tek bir hesapla farklı akademik programlara başvurmasına olanak tanıyabilir. Diğer devlet kurumları ile daha fazla entegrasyon, doğrulama sürecini iyileştirebilir.

VIII. REFERANSLAR

Kitaplar ve Makaleler:

[1] KOU, KOÜ Atama Yönetmeliği

Belgeler:

[2] React.js Dokümantasyonu <https://reactjs.org/docs>.

[3] Twilio API Dokümantasyonu. <https://www.twilio.com/docs>.

[4] Brevo API Dokümantasyonu <https://developers.brevo.com/>

[5] Amazon S3 Dokümantasyonu <https://docs.aws.amazon.com/s3/>

Araçlar ve Kütüphaneler:

[6] Express.js Dokümantasyonu <https://expressjs.com>.

[7] MongoDB Dokümantasyonu. <https://www.mongodb.com/docs/>